An Energy-Efficient Synchronization Scheme Using Exponential Moving Average-Based Drift Compensation and Adaptive Reception Window for Wireless Sensor Networks*

Yeon-woo Lee, Su-ji Park and Il-Gu Lee
[1] Sungshin. Women's University, Seoul, Republic of Korea
{620266012, 620266008, iglee}@sungshin.ac.kr

Abstract

Wireless sensor networks (WSNs) use long sleep intervals for low-power operation. However, accumulated clock drift during these periods leads to beacon reception failures and additional resynchronization overhead, thereby degrading both energy efficiency and reliability. Existing methods rely on fixed symmetric reception windows, which fail to capture the actual distribution characteristics of clock drift. Consequently, reception windows are typically configured based on the maximum drift rate, resulting in unnecessary listening overhead. To address these limitations, this paper proposes a synchronization scheme that combines Exponential Moving Average (EMA)-based drift pre-compensation with a residual statistics-based adaptive reception window. The proposed scheme predicts the drift accumulated during sleep intervals from historical observations and compensates for it at the wake-up time. It then dynamically determines the center and size of the reception window by continuously tracking the mean and variance of the residual error after compensation. Simulation results based on real measurement data collected using Raspberry Pi 5 show that the proposed scheme achieves an average reduction of over 90% in reception window size and approximately 83% in daily energy consumption compared to Time Slotted Channel Hopping (TSCH). In addition, it maintains a stable synchronization success rate of around 91%, demonstrating a clear improvement in reliability compared to TSCH, which achieves only about 50%.

* Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 27, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

1 Introduction

Wireless sensor networks (WSNs) have rapidly advanced and emerged as a core technology in various fields such as defense, environmental monitoring, smart cities, and industrial automation [1]. A WSN is a distributed system in which multiple sensor nodes are wirelessly connected to collect and share data [2]. It operates based on the IEEE 802.15.4 standard, which is designed for low energy consumption and efficient communication [3]. In this standard, time synchronization and duty cycling are managed using beacon signals periodically transmitted by each sensor node, thereby improving the overall power efficiency of the network [4][5].

Duty cycling refers to the operation in which a sensor node periodically alternates between active and sleep modes [6]. The low-power maintenance during sleep mode and the periodic wake-up mechanism are key elements that minimize energy consumption and enable stable long-term operation [7]. Particularly in environments where sensor data are fused based on time information or slot-based communication scheduling is applied, time synchronization accuracy among nodes critically affects system performance and reliability.

However, in real-world environments, the wake-up times of sensor nodes frequently fail to align perfectly. This is caused by clock drift [8], which arises from frequency deviations in the internal oscillators of the sensors, and as the reference time of each node progresses differently, time errors accumulate.

Such clock drift causes temporal mismatches when combining data between sensor nodes, and on the receiving side, data do not arrive at the scheduled time, leading to an increased reception failure rate. As a result, the network performs a resynchronization procedure, which consumes additional energy. Therefore, clock drift acts as a major factor that negatively affects both energy efficiency and data reliability.

Moreover, this problem is further exacerbated by the structural characteristics of IEEE 802.15.4-based WSNs. The standard includes long sleep intervals for energy conservation, making it difficult to detect or compensate for the clock drift accumulated during sleep mode in real time [9]. As a result, when the drift among nodes accumulates beyond a certain level during sleep intervals, sensor nodes are more likely to miss beacon signals, leading to difficulties in maintaining network synchronization.

In addition, WSNs must simultaneously satisfy conflicting requirements such as low power, reliability, low cost, and miniaturization [10]. Increasing the communication frequency or employing high-precision clocks to improve synchronization accuracy increases power consumption and cost, whereas prioritizing energy efficiency degrades synchronization performance. Particularly in environments where maintenance is difficult, such as large-scale battery-powered sensor networks operating over long periods, medical implantable devices, and embedded sensors, energy efficiency and reliability become even more critical [11]. Therefore, approaches that focus solely on synchronization accuracy are inherently limited, and a design that considers both energy efficiency and reliability is needed.

Existing synchronization methods set a fixed reception window centered on the expected arrival time of the data to mitigate timing uncertainty caused by clock drift. The size of this reception window is typically determined based on the maximum drift rate of the clock and the synchronization period [12]. Although this worst-case approach guarantees system stability, it suffers from the following limitations.

First, although the actual clock drift varies across nodes and does not always approach the maximum drift rate, configuring the window based on the fixed maximum value results in an unnecessarily large reception window being maintained, leading to energy waste. Second, while existing methods set the reception window symmetrically around the expected arrival time, the actual drift may accumulate in a particular direction, biasing beacon arrival times toward one side. This leads to an inefficiency in which reception success rates are not sufficiently ensured despite increased energy consumption.

Therefore, in distributed WSN environments, a synchronization method that accounts for clock drift's time-varying nature is needed, going beyond the fixed maximum-value-based approach.

Accordingly, this paper proposes a synchronization scheme that combines Exponential Moving Average (EMA)-based drift pre-compensation with an adaptive reception window. The proposed scheme calculates the EMA-based on drift values observed before a node enters sleep mode to predict future drift. It then pre-compensates for the drift at the wake-up time, thereby reducing accumulated error. In addition, the residual drift remaining after compensation is updated at every synchronization period to dynamically adjust the size and center of the reception window, thereby reducing unnecessary listening time and improving the reception success rate.

By accounting for the time-varying nature of clock drift, the proposed scheme improves both energy efficiency and synchronization reliability over conventional fixed-window approaches, making it an effective time synchronization scheme for energy-constrained WSNs.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the proposed drift-aware adaptive synchronization scheme. Section 4 describes the experimental setup and provides performance analysis. Finally, Section 5 concludes the paper.

2 Related Work

This section analyzes prior studies on time synchronization techniques in WSNs.

Yang et al. [8] proposed BETS, a time synchronization scheme for WSNs based on Bayesian estimation. BETS operates by combining prior measurements of synchronization error with timestamps collected during actual operation to calculate compensation values and has been shown to reduce accumulated error by approximately 53% compared to existing methods while also reducing the number of message exchanges. However, this method focuses solely on error compensation during timestamp exchange and does not account for drift accumulated during sleep mode. Since detecting or compensating for drift in real time during sleep intervals is inherently difficult, longer sleep intervals result in higher beacon reception failure rates and additional energy consumption for resynchronization. Therefore, further research is needed to extend sleep intervals in WSN environments for low-power operation.

Galante-Sempere et al. [13] proposed a method that adds a dedicated low-power Wake-up Receiver (WuR) to sensor nodes, enabling them to detect wake-up signals even while asleep. This architecture mitigates synchronization issues during sleep intervals by minimizing node active time and achieves a sensitivity of −62 dBm with a low power consumption of 528 nW. However, this approach requires additional hardware such as dedicated circuits and pattern recognizers, which may limit its applicability in WSN environments where low cost and miniaturization are required. Furthermore, it has the limitation of requiring hardware modifications to existing commercial systems.

Wang et al. [14] proposed Distributed Sensor Time Synchronization (DSTS), a distributed time synchronization architecture. DSTS uses a Bayesian filter to compensate for nonlinear errors that occur during communication, and leverages Deep Deterministic Policy Gradient (DDPG), a reinforcement learning technique, to allow each node to autonomously adjust its synchronization weights based on the clock stability and link quality of neighboring nodes. This method achieves a high synchronization accuracy on the order of 10^{-10} in networks with heterogeneous sensors. However, the computational and memory overhead of the DDPG agent is too high for direct deployment on low-power sensor nodes. In addition, since the Bayesian filter's statistical model requires prior calibration and the agent requires pre-training, the method has limited immediate applicability to new environments. Moreover, because the reception window size is fixed regardless of channel conditions, excessive listening beyond the actual error may occur, leading to unnecessary energy consumption.

Chang et al. [15] proposed an adaptive time synchronization method for Time Slotted Channel Hopping (TSCH) networks. This method estimates the relative drift between nodes using the ratio of the previously measured time offset to the elapsed time and dynamically adjusts the resynchronization interval by compensating the clock at every slot. As a result, it maintained a synchronization accuracy of less than 76 μs in a 3-hop network while reducing the number of resynchronization packets by up to 83% compared to existing methods. However, since the drift estimation relies only on the most recent measurement, it is sensitive to measurement noise and instantaneous environmental changes and assumes a constant-temperature environment. In addition, when temperature variations occur, resynchronization using a separate sensor is required. Furthermore, since the reception window size is fixed based on the maximum drift rate, unnecessary listening time occurs even when the actual drift between nodes is smaller than the estimated value.

Although the prior studies analyzed above have contributed to improving drift compensation accuracy or reducing the number of packet exchanges, they commonly share the following limitations. First, most methods focus only on drift compensation when packets are exchanged and fail to address the drift accumulated during sleep intervals. Second, since the reception window size is fixed, unnecessary listening continuously occurs regardless of the actual drift distribution. Furthermore, approaches that rely on additional hardware or high-performance computation are not suitable for low-cost and low-power WSN environments.

Therefore, this paper proposes a new synchronization scheme for low-power WSN environments. The proposed scheme operates only in software without additional hardware, predicts and pre-compensates for the drift accumulated during sleep intervals, and dynamically adjusts the reception window based on the statistics of measured residual errors. Through this, the limitations of existing methods can be effectively overcome.

3 Drift-Aware Adaptive Synchronization

The proposed scheme consists of an EMA-based drift prediction and pre-compensation method and an adaptive reception window mechanism that dynamically determines the reception window based on residual statistics. In conventional TSCH environments, a fixed-size reception window is used, determined by the sleep interval and the maximum drift rate of the clock oscillator, and is generally approximated as follows [13][16].

$$W = 2 \cdot ppm \cdot T \tag{1}$$

While this approach ensures synchronization stability through a conservative design, it has the limitation of causing unnecessary energy consumption because it does not reflect the actual drift characteristics.

The proposed scheme separates drift into two components for processing. First, some components exhibit a relatively consistent trend, such as the inherent ppm of the oscillator. For these, the accumulated drift during the next sleep interval is predicted from the drift observed in previous synchronization intervals and then pre-compensated. In contrast, other components vary randomly, such as system jitter, making individual values difficult to predict. These are modeled statistically, and the reception window size is dynamically determined by the model.

3.1 Exponential Moving Average-Based Drift Compensation

Let R_{inst} denote the instantaneous drift rate observed at each interval i. The drift estimate R_{est} is then calculated as in Eq. (2).

$$\boldsymbol{R_{est}[i] = \alpha \cdot R_{inst} + (1-\alpha) \cdot R_{est}[i-1]} \quad (2)$$

If R_{inst} is used directly, instantaneous noise or transient variations directly affect the estimate, which can reduce the stability of compensation. To mitigate this, this study applies EMA, which has low computational complexity, to compute a weighted combination of past observations and the most recent one, thereby capturing both the long-term drift trend and short-term variations simultaneously.

$\alpha \in (0,1)$ is a weighting factor that determines the influence of the most recent observation. A larger αmakes the estimate respond more sensitively to the most recent observation, enabling faster tracking of drift changes, but it also makes the estimate more vulnerable to short-term variations. Conversely, a smaller α yields a more stable estimate, but it also slows the response to drift changes. In this paper, considering that the relative drift between nodes varies across intervals, the stability of the estimate is prioritized, and $\alpha = 0.3$ is used.

The updated R_{est} is used to predict the expected drift that will accumulate during the next sleep interval T. Before entering sleep mode, each node adjusts its wake-up time by $R_{est} \cdot T$ to pre-compensate for the drift that will occur during the sleep interval. As a result, when the next synchronization interval begins, the predicted drift is already reflected, and the beacon is guided to arrive near the center of the reception window.

3.2 Adaptive Receive Window

EMA-based pre-compensation removes predictable components with a consistent trend, but randomly varying components, such as system jitter, remain as residuals even after compensation. These residuals represent how much the beacon arrival time deviates from the expected time at each synchronization interval. To address the limitations of the conventional symmetric fixed window, this section proposes a method that dynamically determines the reception window by tracking the bias and distribution width of the drift residuals. For the residual r observed at each synchronization interval i, the mean μ_r and variance σ_r^2 of the residuals are calculated as in Eq. (3) and (4).

$$\boldsymbol{\mu_r[i] = \alpha \cdot r + (1-\alpha) \cdot \mu_r[i-1]} \quad (3)$$

$$\sigma_r^2[i] = \alpha \cdot (r - \mu_r[i])^2 + (1-\alpha) \cdot \sigma_r^2[i-1] \quad (4)$$

μ_r and σ_r are used to determine the center position and size of the reception window, respectively. μ_r represents the average bias of the residuals, based on which the window center is shifted in the corresponding direction. In contrast, σ_r represents the spread of the residual distribution, based on which the window size is determined. By separately determining the center position and size of the window in this way, even when the drift is skewed in a specific direction, the reception window can be maintained at the minimum size without unnecessarily extending the window in the opposite direction. In addition, μ_r and σ_r are statistics of only the residuals remaining after the predictable components are removed through pre-compensation. Therefore, they are used to dynamically determine the reception window. Based on this, the reception window size for the next synchronization interval is set as in Eq. (5).

$$\boldsymbol{W[i+1] = |\mu_r[i]| + 2k \cdot \sigma_r[i]} \quad (5)$$

k is a coefficient corresponding to the target confidence level. Assuming a Gaussian distribution, k is set such that the probability of the residuals falling within the range of $\pm k\sigma_r$ matches the target confidence level. For example, to target a synchronization success rate of 99% between two nodes, k can be set to approximately 2.81. However, this correspondence relies on the assumption that the

residuals follow a Gaussian distribution. If the actual residual distribution deviates from this assumption, the achieved synchronization success rate may differ from the intended target. Nevertheless, the proposed scheme narrows the reception window when the residuals are small and expands it when the residuals are large, thereby minimizing unnecessary listening time while ensuring reliable beacon reception.

The EMA-based pre-compensation and residual-based adaptive reception window described above are performed repeatedly at every synchronization interval. The overall operational flow is shown in Algorithm 1.

Algorithm 1. Per-round operation of the proposed method

Input: α − EMA weight
k – reliability coefficient (e.g., k≈2.81 for 99%)
T – sleep period

Initialize: $R_{est} \leftarrow 0, \mu_r \leftarrow 0, \sigma_r{}^2 \leftarrow 0$

1**: for** each round i **do**
2: $W[i] \leftarrow |\mu_r| + 2k \cdot \sigma_r$
3: Receive beacon within $W[i]$
4: $r \leftarrow (R_{inst} - R_{est}) \cdot T$
5: Update $\mu_r, \sigma_r{}^2, R_{est}$ via EMA
6: Adjust wake-up time: $t_{next} + R_{est} \cdot T$
7: Sleep
8: end for

4 Experiments and Analysis

4.1 Experimental Setup

To evaluate the effectiveness of the proposed scheme, two Raspberry Pi 5 devices were placed in the same environment, and clock data were collected over 44 hours. One device was configured as an access point (AP) to provide the reference time, while the other was configured as a station to measure the relative clock drift between the two nodes. The measurement results showed an average drift rate of 490 ppm, which significantly exceeds the typical tolerance range of approximately ±20 to 100 ppm for low-cost oscillators [17]. This discrepancy arises from the jitter caused by the scheduling characteristics of the Linux-based operating system on the Raspberry Pi, which are captured in the measurements. However, the purpose of this experiment is not to measure the absolute drift value, but to analyze the relative performance difference between TSCH and the proposed scheme under the same drift conditions. Therefore, the fact that the measured drift values are somewhat higher than the actual hardware-level drift does not affect the performance comparison of the proposed scheme. Moreover, since dedicated WSN oscillators typically exhibit significantly lower drift rates in the range of approximately 20–100 ppm, the residual variance after EMA compensation is expected to be substantially smaller in practice. Consequently, the proposed scheme is expected to achieve narrower reception windows and lower energy consumption while maintaining the target synchronization success rate in actual WSN deployments.

Based on the measured drift data, Python simulations were performed on Google Colab to evaluate the performance of the proposed scheme. The performance metrics used were the reception window size, the synchronization success rate, and the energy consumption. The sleep interval T_{sleep} was varied from 10 s to 500 s, and the performance was evaluated for each value. The energy consumption was calculated using an energy model that accounts for the major factors affecting energy consumption within a synchronization interval, and the specific model is presented in Section 4.2.

The fixed reception window approach of TSCH was used as the baseline. TSCH is a standard synchronization protocol widely used in IEEE 802.15.4-based low-power wireless networks, incorporating various mechanisms such as channel hopping and slot scheduling. In this study, only the method for determining the reception window size was compared. The reception window of TSCH is determined by Equation (1), and the previously measured value of 490 ppm was applied in this experiment. Since this approach determines the window size based on the maximum drift rate without considering the actual drift distribution, the same calculation method is applied across all sleep intervals.

In contrast, as described in Section 3, the proposed scheme first removes predictable drift components through EMA-based pre-compensation and then dynamically determines the reception window size at every synchronization interval based on the residual statistics μ_r and σ_r. This enables adaptive synchronization that accounts for the actual drift characteristics.

4.2 Energy Consumption Modeling

This section presents the system energy consumption model introduced in Section 4.1. The energy consumed during one synchronization interval, E_{cycle}, is expressed as in Eq. (5).

$$E_{cycle} = P_{sleep} \cdot T_{sleep} + E_{wakeup} + P_{RX} \cdot W + E_{resync} \tag{5}$$

In Equation (5), each term represents a specific energy component: $P_{sleep} \cdot T_{sleep}$ denotes the baseline energy consumed during the sleep interval; E_{wakeup} denotes the energy consumed during the wake-up process, including oscillator operation and MCU initialization; $P_{RX} \cdot W$ denotes the energy consumed during the reception window maintenance period; and E_{resync} denotes the energy consumed for resynchronization when synchronization fails. By extending this over the total operating time T_{total}, the daily energy consumption E_{sys} (in J/day) can be expressed as in Eq. (6).

$$E_{sys} = P_{sleep} \cdot T_{sleep} + \frac{T_{total}}{T_{sleep}}[E_{wakeup} + P_{RX} \cdot W + (1 - p^2{}_{SR}) \cdot E_{resync}] \tag{6}$$

In Equation (6), the terms that recur at every interval — E_{wakeup}, $P_{RX} \cdot W$, and E_{resync}— are scaled by the number of synchronization intervals, T_{total}/T_{sleep}. In contrast, since sleep energy is consumed continuously regardless of the synchronization interval, the $P_{sleep} \cdot T_{sleep}$ term is calculated separately. Furthermore, since resynchronization occurs only when synchronization fails, not at every interval, it is modeled as an expected value that incorporates the failure probability. Assuming that each of the two sensor nodes synchronizes successfully with probability P_{SR}, the probability that at least one node fails to synchronize is $1 - P_{SR}^2$. Accordingly, the resynchronization energy per interval can be expressed as $(1 - P_{SR}^2) \cdot E_{resync}$.

Although the drift data in this study were collected using Raspberry Pi 5 devices, the energy consumption analysis was conducted to reflect actual WSN deployment conditions. Accordingly, for the energy model parameters, the values of the TI CC2538 low-power wireless communication chip [18], widely used as the reference hardware in TSCH-related studies, were adopted. Specifically, $P_{RX} =$

60 mW, $P_{sleep} = 4$ μW, $E_{wakeup} = 0.5$ mJ, and $E_{resync} = 30$ mJ were used, and all parameters are summarized in Table 1.

Table 1. Energy consumption model parameters based on TI CC2538

Parameter	Value	Description
P_{RX}	$60\ mW$	Receive power
P_{sleep}	$4\ \mu W$	Sleep power
E_{wakeup}	$0.5\ mJ$	Wake-up overhead
E_{resync}	$30\ mJ$	Resynchronization penalty
P_{SR}	99 %	Target synchronization success rate

4.3 Performance Evaluation and Analysis

This section compares the performance of TSCH and the proposed scheme in terms of the reception window size, the synchronization success rate, and the daily system energy consumption. First, the reception window size determined by each method and the synchronization success rate achieved with that window are analyzed. This enables a comparison of the reception window configuration approach and the resulting performance difference under the same drift conditions.

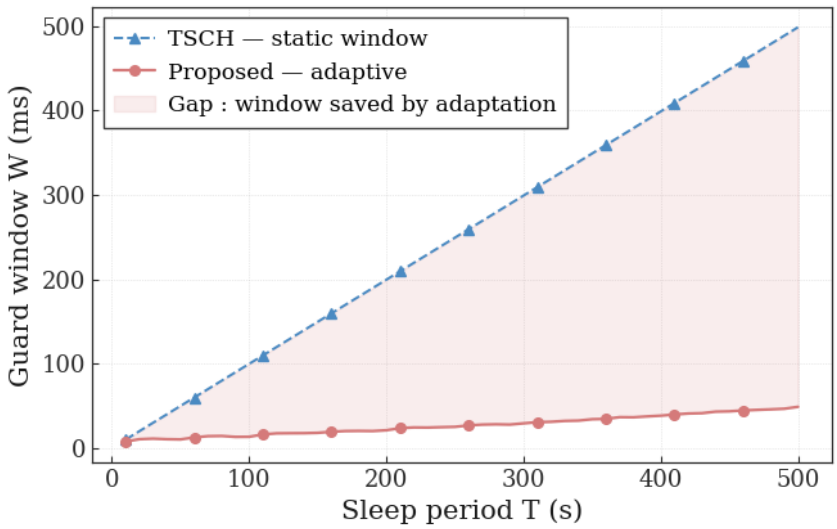


Figure 1. Guard window size computed by each method

Figure 1 shows the reception window size of each method as a function of the sleep interval T_{sleep}. For TSCH, the window size increases linearly with T_{sleep} according to $W_{TSCH} = 2 \cdot ppm \cdot T_{sleep}$, reaching approximately 500 ms at $T_{sleep} = 500$ s. This reveals the limitation of TSCH, which determines the window based on the maximum drift rate and therefore consistently maintains a conservatively large window at all times, regardless of the actual drift. In contrast, the proposed scheme operates with a reception window of less than 50 ms under the same conditions. This is because the

predictable drift component is removed through EMA-based pre-compensation, allowing synchronization to be maintained with a narrow window proportional to the residual distribution width σ_r. Consequently, the gap in window size difference between the two methods widens sharply as T_{sleep} increases, indicating that the proposed scheme becomes increasingly efficient in low-power WSN environments with long sleep intervals.

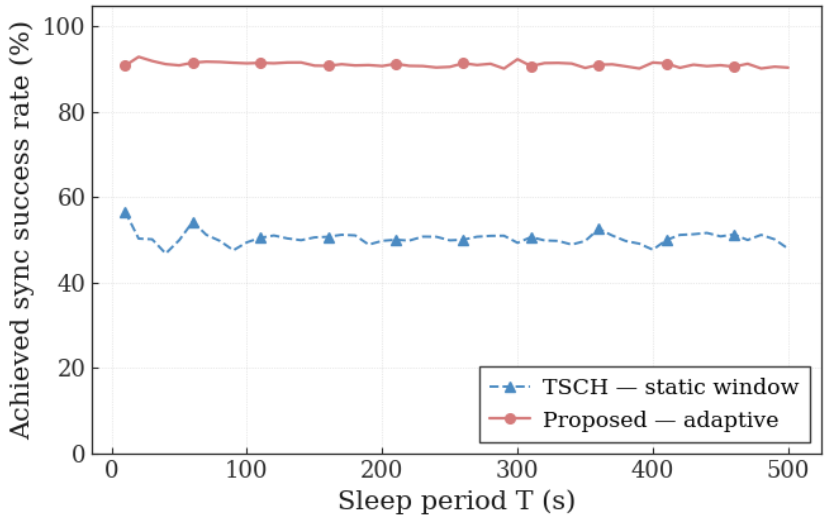


Figure 2. Resulting synchronization success rate per method

Figure 2 compares the measured synchronization success rates obtained by applying the reception windows derived for each method in Figure 1. Although TSCH employs a larger window and is therefore expected to achieve a high synchronization success rate, the observed performance stayed around 50% across all sleep intervals. This stems from TSCH's window configuration approach. Since TSCH centers the window symmetrically on the average drift rate, drift values larger than the mean fall outside the window. In particular, when the drift is skewed in one direction — as observed in this experiment — beacon losses are concentrated on one side. As a result, a substantial proportion of beacons fail to be received. This indicates a fundamental limitation of TSCH: it cannot guarantee the synchronization success rate without accounting for the distribution characteristics of the drift, even when the window size is set sufficiently large.

In contrast, the proposed scheme achieved a stable average synchronization success rate of approximately 91% across all sleep interval conditions, despite using a narrower window than TSCH. This results from effectively eliminating losses caused by drift bias by aligning the window center with the residual mean and sizing the window proportionally to σ_r. In summary, Figures 1 and 2 clearly illustrate how the window configuration approach affects performance. Despite using a conservatively large window, TSCH yields a low success rate, whereas the proposed scheme sustains a high success rate with a much smaller window.

However, since the two methods yield different synchronization success rates, a direct comparison of their energy efficiency is not straightforward. To address this, an additional experiment was conducted in which the TSCH reception window was reconfigured to match the success rate of the proposed scheme. Since the standard TSCH design equation does not directly account for the target success rate, this study adjusts the window based on the empirical quantile of the measured absolute drift values. Specifically, given the target success rate P_{SR}, the reception window is defined as follows.

$$W_{TSCH} = 2 \cdot Q_{P_{SR}} \cdot (|drift|) \quad (7)$$

$Q_{P_{SR}}$ denotes the P_{SR}-quantile of the absolute drift distribution. This ensures that both TSCH and the proposed scheme operate at the same success rate. Under this condition, the reception window sizes required by the two methods are compared in Table 2.

Table 2. Guard window size comparison at matched synchronization success rate

Sleep period T	*W (TSCH)*	*W(proposed)*	*Reduction*
30s	34.3ms	9.4ms	72.6%
60s	64.7ms	10.6ms	83.6%
120s	126.1ms	14.7ms	88.3%
300s	308.6ms	24.7ms	92.0%
500s	513.0ms	41.4ms	91.9%

As shown in Table 2, even when the TSCH window is reconfigured to maintain the same reliability, the limitation that the reception window size grows in proportion to T_{sleep} remains. This is because TSCH does not account for the statistical distribution or variability of the drift and sets the window based on the maximum drift. In contrast, since the reception window of the proposed scheme depends on the residuals remaining after pre-compensation, the window size grows very slowly even as T_{sleep} increases. As a result, an average reduction of more than 90% in the window size is observed in low-power environments with sleep intervals of 100 s or longer.

A reduction in the reception window size implies a shorter reception time. However, the actual energy efficiency of a sensor must consider not only the reception operation but also the overhead associated with sleep, wake-up, and resynchronization failures. Accordingly, the following analysis compares the energy consumption of the two methods under the condition that both achieve the same synchronization success rate. For this purpose, the window sizes in Table 2 are substituted into the E_{sys} model presented in Section 4.2 to calculate energy consumption, and the results are shown in Figure 3.

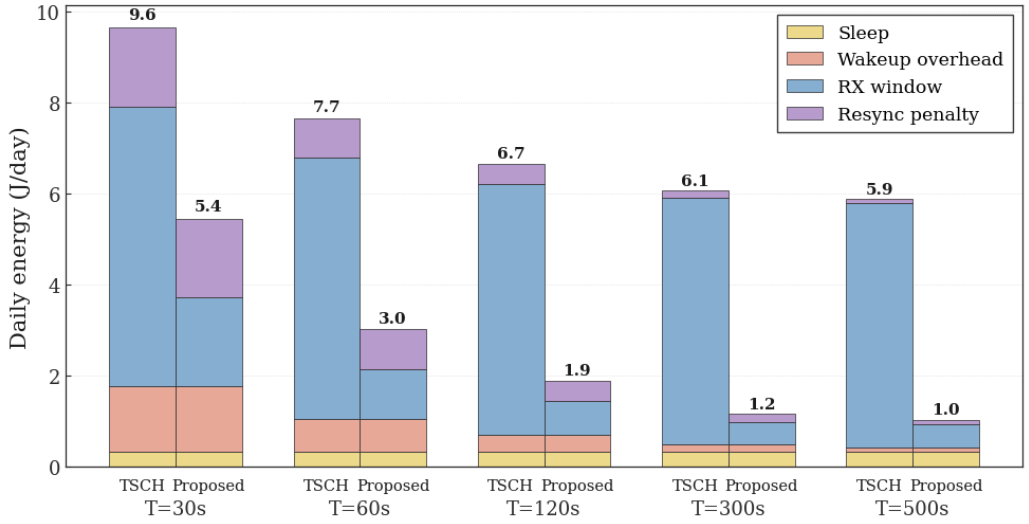


Figure 3. Per-component daily energy consumption under comparable synchronization success rates

Figure 3 shows the composition of daily energy consumption as a function of the sleep interval T_{sleep}. Since the baseline power consumed during sleep and the wake-up overhead are independent of the reception window size, they appear identically for both methods. In contrast, the reception energy is proportional to the window size, and thus it decreases significantly with the reduction in the window size in the proposed scheme. As a result, the proposed scheme consumed less energy than TSCH under all sleep interval conditions, achieving an energy reduction of up to approximately 83%. Particularly at $T_{sleep} = 500$ s, TSCH consumed 5.9 J/day, whereas the proposed scheme consumed only 1.0 J/day, clearly demonstrating the energy reduction effect.

A notable observation is that the energy reduction rate tends to increase as the sleep interval becomes longer. This is due to the structural difference that the window size of the proposed scheme is barely affected by the increase in T_{sleep}, whereas that of TSCH increases linearly with T_{sleep}. Therefore, the advantages of the proposed scheme become more pronounced in low-power WSN environments that use long sleep intervals.

However, this result does not imply that extending the sleep interval indefinitely is always beneficial. As shown in Figure 3, the daily energy consumption of the proposed scheme converges to approximately 1.2 J/day near $T_{sleep} = 300$ s, and even when T_{sleep} is further increased to 500 s, the additional reduction is only about 0.2 J/day. In other words, there exists a region beyond which the energy reduction effect diminishes.

Conversely, when T_{sleep} is too short, the advantages of the proposed scheme are not fully realized. As shown in Table 2, at $T_{sleep} = 30$s, the window reduction rate remains at 72.6%, and a stable window reduction of more than 90% on average compared to TSCH is achieved only in the range of $T_{sleep} \geq 100$ s. In terms of energy as well, at $T_{sleep} = 30$ s, the proposed scheme consumes 5.4 J/day, which is approximately 2.8 times the consumption at $T_{sleep} = 120$ s. This is because at short sleep intervals, the number of wake-up per unit time increases, causing repetitive overhead energy to accumulate.

In addition, in actual WSN environments, not only energy efficiency but also factors such as response delay, data transmission interval, and the delay incurred during resynchronization must be considered together. These factors generally tend to become unfavorable as the sleep interval becomes longer. For example, in applications that require real-time responsiveness, such as environmental monitoring, an excessively long sleep interval can degrade the quality of service. Therefore, maximizing the sleep interval based solely on energy reduction is not desirable; rather, a practical design direction is to find a balance between the marginal utility of the reduction and the response delay requirements.

Based on this overall analysis, the range of $T_{sleep} = 100$–300 s is judged to be a practical operating range that balances energy efficiency and response delay under the experimental conditions. Particularly, at $T_{sleep} = 300$ s, the energy consumption reaches approximately 1.2 J/day, with a window reduction of about 92% compared to TSCH. At the same time, in terms of response delay, an interval within 5 minutes satisfies a practical level. This indicates that, when energy efficiency, synchronization performance, and response delay are all considered together in this experimental environment, $T_{sleep} = 300$ s represents the most balanced operating point. However, this optimal point may be adjusted according to the environment and requirements of the WSN, and the results of this study suggest that the proposed scheme achieves substantial performance improvement across various operating conditions.

5 Conclusion

This paper proposed a synchronization scheme that combines EMA-based drift pre-compensation with an adaptive reception window to address the problems of energy waste and degraded synchronization reliability caused by clock drift in low-power WSN environments.

Experimental results using measured data obtained from Raspberry Pi 5 devices show that the proposed scheme achieves an average reception window reduction of more than 90% and an energy saving of approximately 83% compared to TSCH. In addition, it maintains a stable synchronization success rate of around 91%, demonstrating a significant improvement in reliability over TSCH, which remains at approximately 50%. These results demonstrate that the proposed scheme can simultaneously improve energy efficiency and synchronization performance by removing the mean drift component and dynamically adjusting the reception window based on the residuals.

However, the Raspberry Pi used in the experiments is a general-purpose computer and has limitations in exactly reproducing the hardware sleep and wake-up operations of MCU-based sensor nodes. As a result, the measured drift included OS scheduling jitter and reached approximately 490 ppm, which is significantly higher than the typical level of around 30 ppm observed in dedicated WSN sensor nodes. Therefore, future work will further validate the performance of the proposed scheme in more realistic environments using dedicated low-power WSN sensors such as the CC2538 and further evaluate its applicability in actual sensor networks. As WSNs increasingly serve as an enabling technology for e-business applications, including smart logistics, supply chain management, and industrial monitoring, the proposed synchronization scheme is expected to support the sustainable and energy-efficient operation of such systems.

Acknowledgements

This work is supported by the Ministry of Trade, Industry and Energy (MOTIE) under Training Industrial Security Specialist for High-Tech Industry [grant number RS-2024-00415520] supervised by the Korea Institute for Advancement of Technology (KIAT), the Ministry of Science and ICT (MSIT) under the ICAN (ICT Challenge and Advanced Network of HRD) program [grant number IITP-2022-RS-2022-00156310] and National Research Foundation of Korea (NRF) grant [RS-2025-00518150], and the Information Security Core Technology Development program [grant number RS-2024-00437252] supervised by the Institute of Information & Communication Technology Planning & Evaluation (IITP).

References

[1] Zhu, Y., & Zhu, J. (2025). Mobile WSN wormhole attack without beacon node localization based on GNDO algorithm. Security and Privacy, 8(1).
[2] Lee, S.-Y., & Wang, J. (2014). Intelligent and load balancing routing method in wireless sensor networks (Patent No. KR101392497B1). Korean Intellectual Property Office. https://patents.google.com/patent/KR101392497B1
[3] Khalifeh, A., Tanash, R., Al Qudah, M., & AlAgtash, S. (2023). Enhancing energy efficiency of IEEE 802.15.4-based industrial wireless sensor networks. Journal of Industrial Information Integration, 33.
[4] Jung, P. S., Kim, H. S., & Oh, Y. H. (2011). Cross layer protocol based on IEEE 802.15.4 for improving energy efficiency. The Journal of Korean Institute of Communications and Information Sciences, 36(7), 669–677.
[5] Kim, S. Y., Yoon, S. W., & Lee, I. G. (2024). An adaptive power-saving mechanism of low-power wake-up receiver against battery exhaustion attacks. Journal of the Korea Institute of Information Security and Cryptology, 34(3), 393–401.
[6] Mohammadi, R., & Shirmohammadi, Z. (2023). DRDC: Deep reinforcement learning based duty cycle for energy harvesting body sensor node. Energy Reports, 9, 1707–1719.
[7] Ruíz-Guirola, D. E., Montejo-Sánchez, S., Leyva-Mayorga, I., Han, Z., Popovski, P., & López, O. L. A. (2026). Energy management and wake-up for IoT networks powered by energy harvesting. IEEE Internet of Things Journal. Advance online publication.
[8] Yang, T., Niu, Y., & Yu, J. (2020). Clock synchronization in wireless sensor networks based on Bayesian estimation. IEEE Access, 8, 69683–69694.
[9] Feng, H., Yang, Z., Shi, Y., & Nabipour, N. (2020). Compensation method for sensor network clock error based on cyclic symmetry algorithm. Sensors, 20(6), 1738.
[10] Alaerjan, A. (2023). Towards sustainable distributed sensor networks: An approach for addressing power limitation issues in WSNs. Sensors, 23(2), 975.
[11] Evangelakos, E. A., Kandris, D., Rountos, D., Tselikis, G., & Anastasiadis, E. (2022). Energy sustainability in wireless sensor networks: An analytical survey. Journal of Low Power Electronics and Applications, 12(4), 65.
[12] Bober, W., & Bleakley, C. J. (2014). BailighPulse: A low duty cycle data gathering protocol for mostly-off wireless sensor networks. Computer Networks, 69, 51–65.
[13] Mavromatis, A., Papadopoulos, G. Z., Elsts, A., Montavont, N., Piechocki, R., Tryfonas, T., Oikonomou, G., & Fafoutis, X. (2019). Adaptive guard time for energy-efficient IEEE 802.15.4 TSCH networks. In Wired/Wireless Internet Communications (WWIC 2019), Lecture Notes in Computer Science (Vol. 11618, pp. 15–26). Springer.
[14] Zorbas, D., Abdelfadeel, K., Kotzanikolaou, P., & Pesch, D. (2020). TS-LoRa: Time-slotted LoRaWAN for the industrial Internet of Things. Computer Communications, 153, 1–10.
[15] Daneels, G., Municio, E., Van de Velde, B., Ergeerts, G., Weyn, M., Latré, S., & Famaey, J. (2018). Accurate energy consumption modeling of IEEE 802.15.4e TSCH using dual-band OpenMote hardware. Sensors, 18(2), 437.
[16] Galante-Sempere, D., Ramos-Valido, D., Lalchand Khemchandani, S., & del Pino, J. (2020). Low-power RFED wake-up receiver design for low-cost wireless sensor network applications. Sensors, 20(22), 6406.
[17] Wang, Z., Lyu, D., Zhou, P., Ge, Y., Hu, Y., Zhu, R., Wang, W., & Yang, X. (2025). Robust high-precision time synchronization for distributed sensor systems in challenging environments. Remote Sensing, 17(22), 3715.
[18] Texas Instruments. (2015). CC2538 powerful wireless microcontroller system-on-chip for 2.4-GHz IEEE 802.15.4, 6LoWPAN, and ZigBee applications (Datasheet SWRS096D).